Arrays Study Guide

Arrays Study Guide

1. Overview

Arrays are the foundation of Data Structures & Algorithms and appear in **over 70% of LeetCode problems**. Mastering arrays means mastering iteration patterns, in-place updates, prefix computations, searching, and optimization techniques.
Arrays are ideal when you need:

- Fast random access (O(1))
- Sequential processing
- Sliding window operations
- Prefix/suffix computations
- Sorting-based strategies

---

2. Core Array Patterns

2.1 Two-Pointer Traversal

Used when the array is sorted or when scanning from both ends.
Examples:

- Two Sum II
- Remove Duplicates
- Reverse String

Key idea:

```
l, r = 0, n-1
while l < r:
    ...
```

2.2 Sliding Window

Used for subarray problems involving sums, counts, or constraints.
Examples:

- Longest Substring Without Repeating Characters
- Minimum Window Substring
- Max Consecutive Ones

Key idea:

```
while window invalid: shrink left
while window valid: expand right
```

2.3 Prefix Sum

Used when computing sums over ranges.
Examples:

- Subarray Sum Equals K
- Range Sum Query

Key idea:

```
prefix[i] = prefix[i-1] + nums[i]
```

2.4 Difference Array

Used for efficient range updates.
Examples:

- Corporate Flight Bookings

Key idea:

```
diff[l] += val
diff[r+1] -= val
```

2.5 Sorting + Greedy

Sorting reveals structure.
Examples:

- Meeting Rooms
- Merge Intervals
- Assign Cookies

2.6 Binary Search on Arrays

Used when array is sorted or monotonic.
Examples:

- Search Rotated Sorted Array
- Find Minimum in Rotated Array
- Koko Eating Bananas (binary search on answer)

---

3. Essential Techniques

3.1 In-Place Modification

Used to reduce space complexity.
Examples:

- Move Zeroes
- Remove Element

Key idea:

```
write = 0
for read in range(n):
    if nums[read] != 0:
        nums[write] = nums[read]
        write += 1
```

3.2 Hash Map + Array

Used when tracking frequencies or indices.
Examples:

- Two Sum
- Subarray Sum Equals K
- Longest Consecutive Sequence

3.3 Kadane's Algorithm (Max Subarray)

```
cur = best = nums[0]
for n in nums[1:]:
    cur = max(n, cur + n)
    best = max(best, cur)
```

---

4. Example Problem: Two Sum

```
def twoSum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target-n], i]
        seen[n] = i
```

Why it works:

- Hash map gives O(1) lookup.

5. Example Problem: Longest Subarray with Sum K (Prefix Sum)

```
def subarraySum(nums, k):
    prefix = 0
    seen = {0: 1}
    count = 0

    for n in nums:
        prefix += n
        if prefix - k in seen:
            count += seen[prefix - k]
        seen[prefix] = seen.get(prefix, 0) + 1

    return count
```

---

6. Common Pitfalls

- Off-by-one errors in prefix sums
- Forgetting to update left pointer in sliding window
- Misusing two pointers on unsorted arrays
- Overlooking negative numbers (breaks fixed-window logic)
- Using binary search on non-monotonic arrays

---

7. Practice Roadmap

Beginner

- Two Sum
- Move Zeroes
- Contains Duplicate
- Maximum Subarray

Intermediate

- 3Sum
- Product of Array Except Self
- Subarray Sum Equals K

- Longest Consecutive Sequence

Advanced

- Minimum Window Substring
- Sliding Window Maximum
- Search Rotated Sorted Array
- Koko Eating Bananas

---

8. Key Takeaways

- Arrays are the backbone of DSA.
- Most array problems follow a small number of patterns.
- Master two pointers, sliding window, and prefix sums.
- Sorting often reveals hidden structure.

End of document.